

บทที่ 2 : กระบวนการ พัฒนาซอฟต์แวร์

แนะนำกระบวนการพัฒนาฯ **01**

Waterfall Model **02**

V model **03**

Rapid Application Development (RAD) **04**

Rational Unified Process (RUP) **05**

Spiral Model **06**

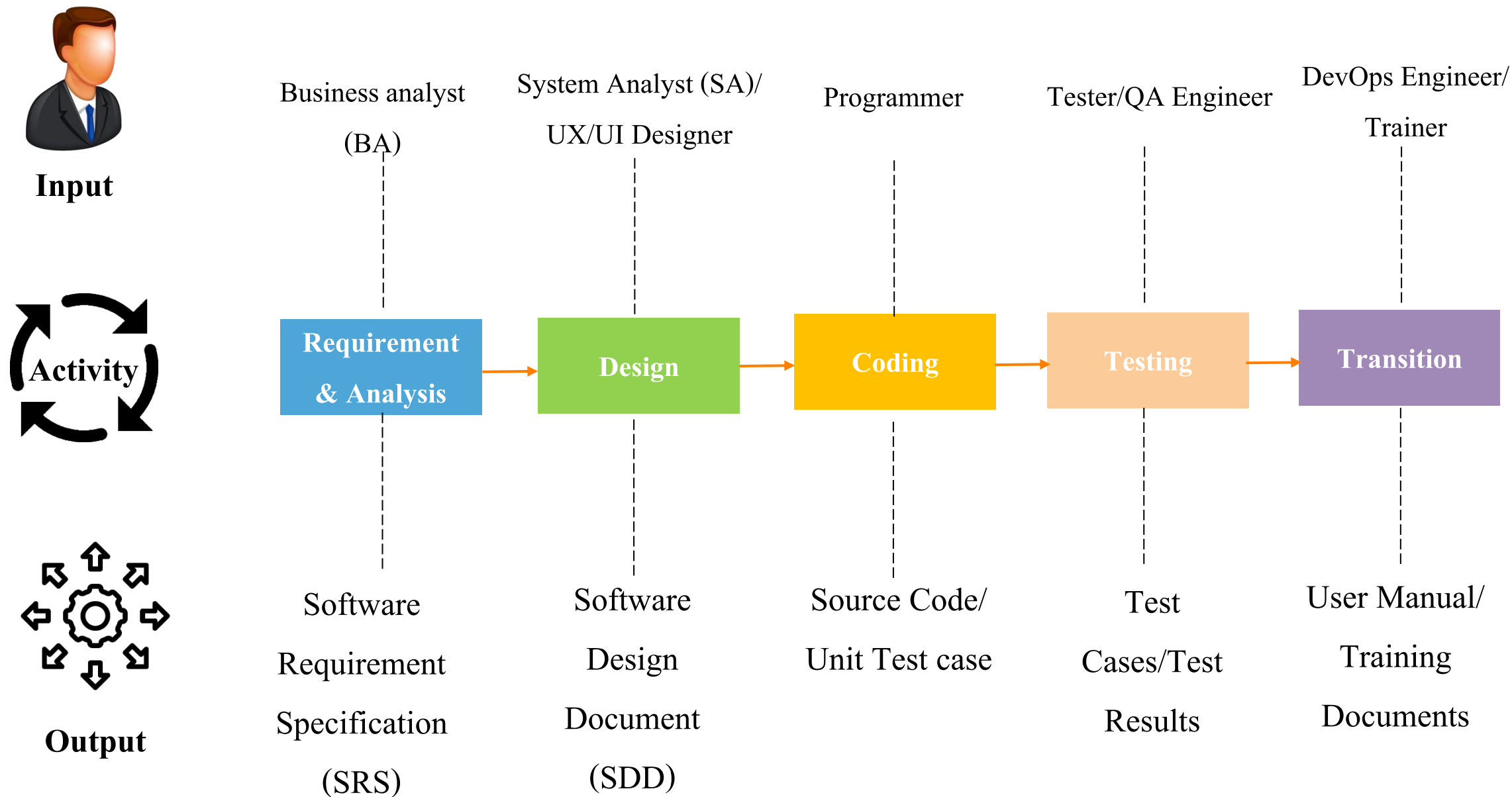
Agile Development **07**

DevOps **08**

1. แนะนำกระบวนการพัฒนาซอฟต์แวร์

1.1 นิยามของกระบวนการพัฒนาซอฟต์แวร์

กระบวนการพัฒนาซอฟต์แวร์ (Software Process) คือ กลุ่มของกิจกรรม (Activity) การพัฒนาซอฟต์แวร์ ซึ่งเกี่ยวข้องกับ คน เครื่องมือ หรือสิ่งอำนวยความสะดวกในการดำเนินงาน (Input) และผลที่ได้รับจากการดำเนินงาน (Output)



ภาพที่ 1-1 : ความสัมพันธ์ระหว่างกิจกรรม อินพุต และเอาต์พุตของการพัฒนาซอฟต์แวร์

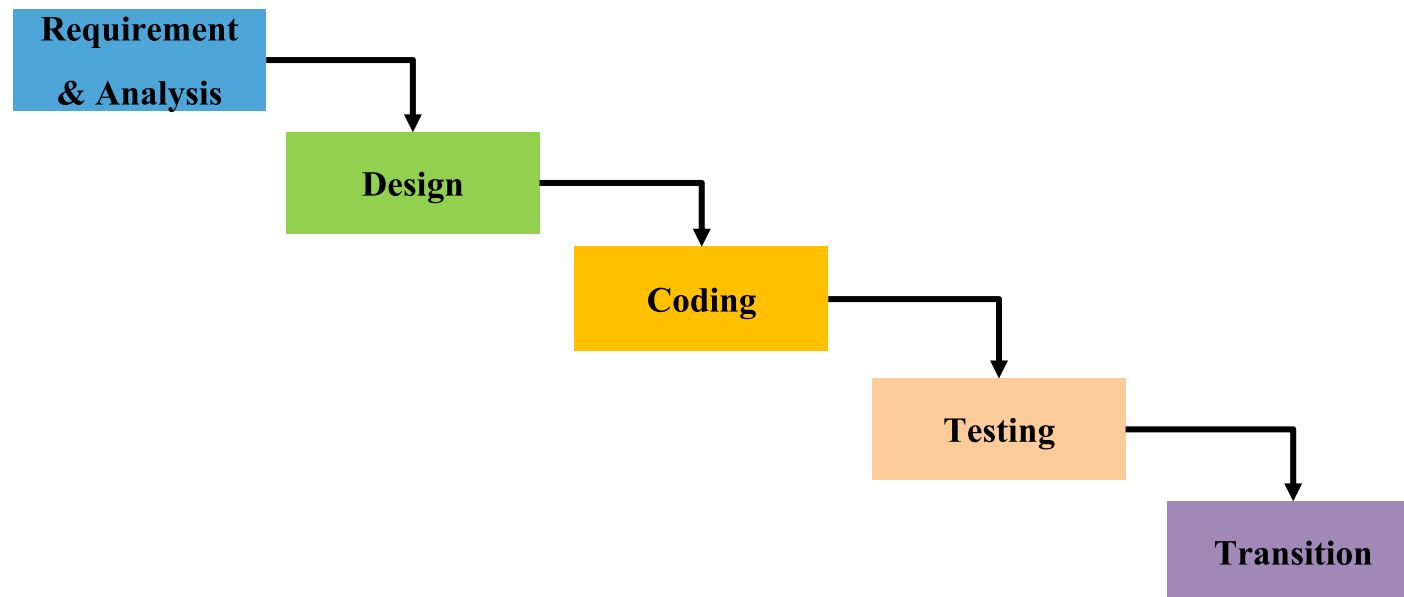
1.2 นิยามของแบบจำลองกระบวนการพัฒนาซอฟต์แวร์

แบบจำลองกระบวนการพัฒนาซอฟต์แวร์ (Software Process Model) คือ แนวคิดของกระบวนการพัฒนาซอฟต์แวร์ ซึ่งแต่ละองค์กรสามารถนำไปปรับใช้ให้เข้ากับการดำเนินการขององค์กรได้ เช่น

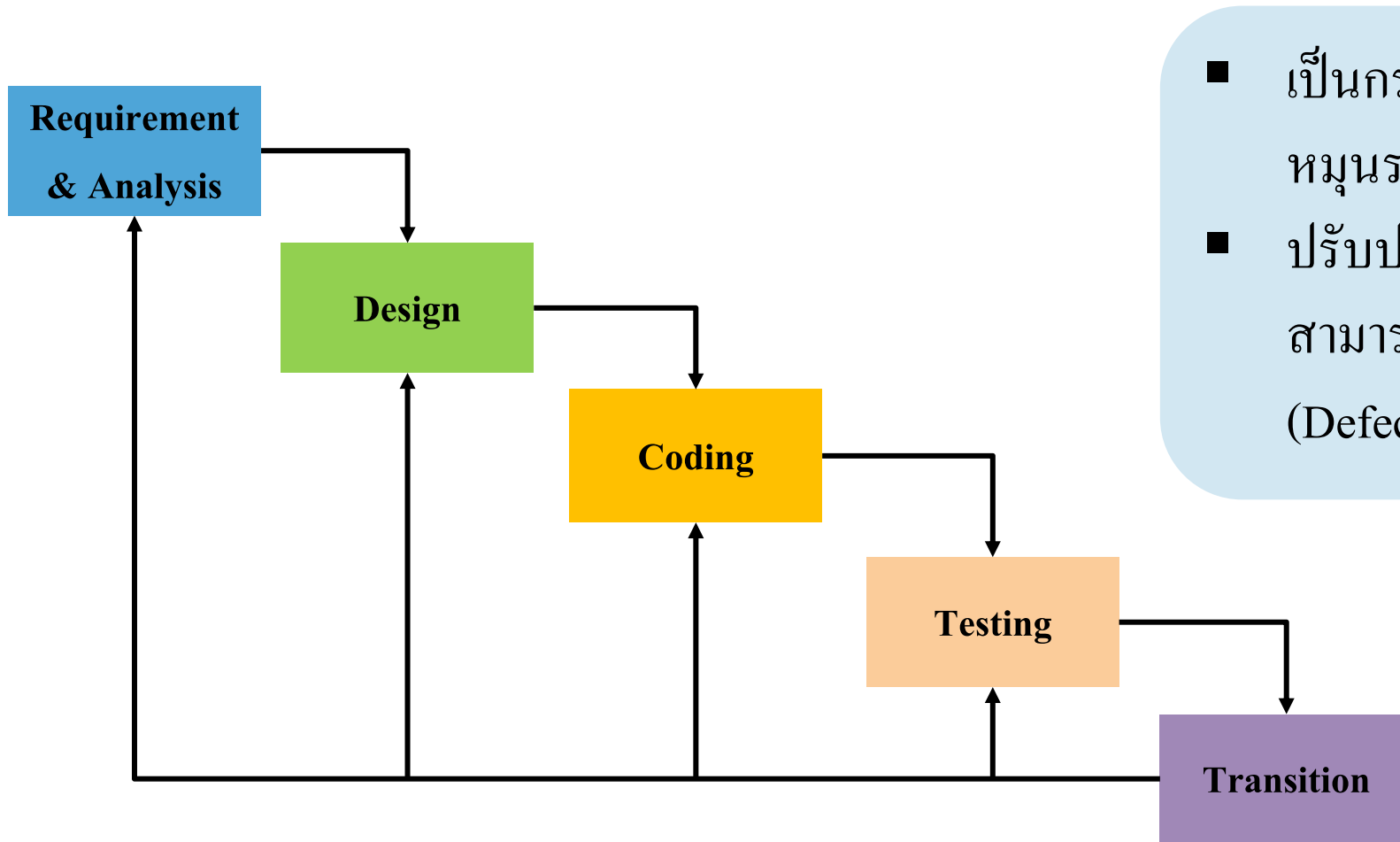
- 1 Waterfall Model
- 2 V model
- 3 Rational Unified Process (RUP)
- 4 Spiral Model
- 5 Rapid Application Development (RAD)
- 6 Agile Development
- 7 DevOps
- 8 Iterative Waterfall Model

2. Waterfall Model

- เป็นกระบวนการพัฒนาซอฟต์แวร์แบบเป็นลำดับขั้นตอน (Sequential Development)
- ดำเนินการให้เสร็จสมบูรณ์ในขั้นนั้น ๆ แล้วค่อยดำเนินการขั้นถัดไป คล้ายน้ำตก (Waterfall)
- ผู้ใช้งานได้เห็นตัวซอฟต์แวร์เมื่อใกล้ขั้นสุดท้ายของโครงการ
- มีเอกสารชัดเจนในแต่ละขั้นตอน เช่น SRS, SDD, Test Case, Manual (Document-driven Dev.)
- เหมาะกับการพัฒนาซอฟต์แวร์ที่มีความต้องการชัดเจน (Plan-driven Dev.)



ภาพที่ 2-1 : Waterfall Model

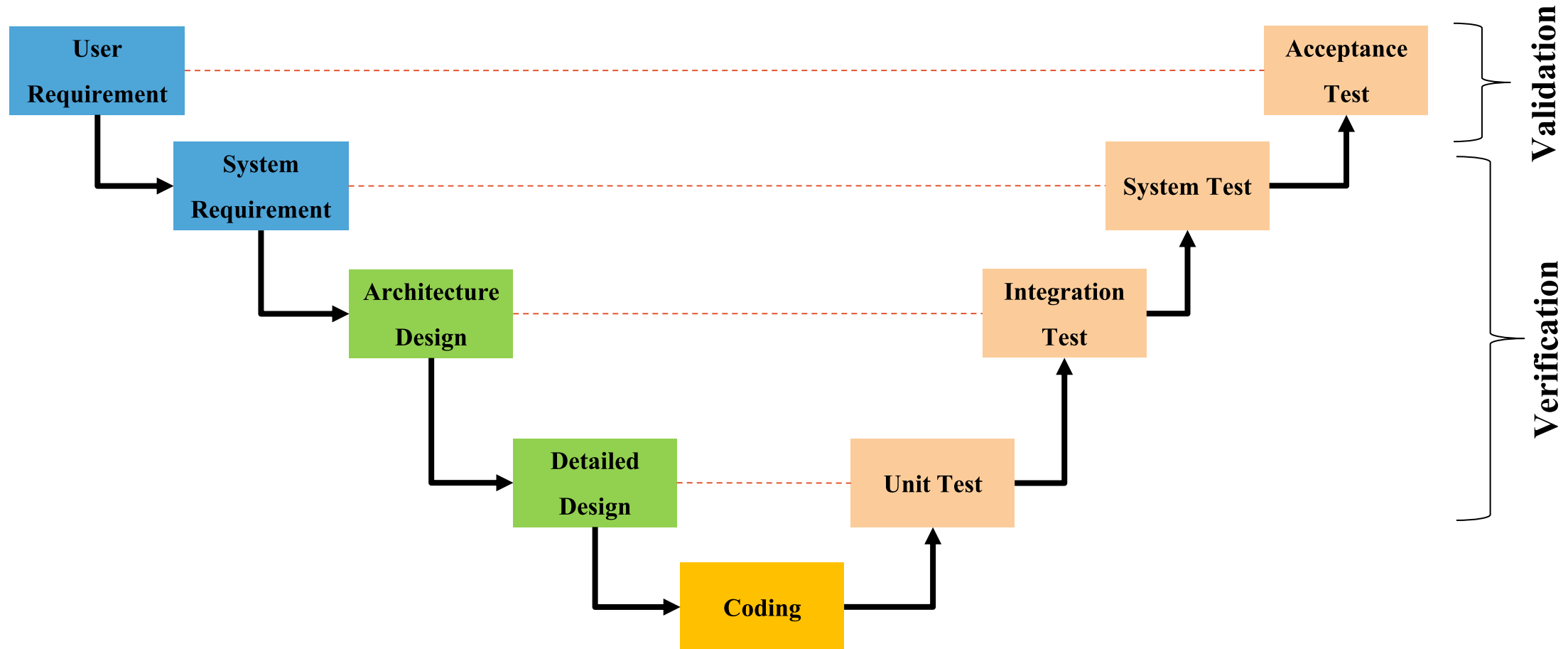


- เป็นกระบวนการพัฒนาซอฟต์แวร์แบบเป็น
หมุนรอบ (**Iterative Development**)
- ปรับปรุงจาก Waterfall Model เพื่อให้
สามารถย้อนกลับมาแก้ไขข้อผิดพลาด
(Defect) ได้

ภาพที่ 2-2 : Iterative Waterfall Model

3. V Model

- เป็นกระบวนการพัฒนาซอฟต์แวร์แบบเป็นลำดับขั้นตอน (Sequential Development)
- ปรับปรุงจาก Waterfall Model เน้นในเรื่องของการทดสอบมากยิ่งขึ้น



ภาพที่ 3-1 : V Model

Verification

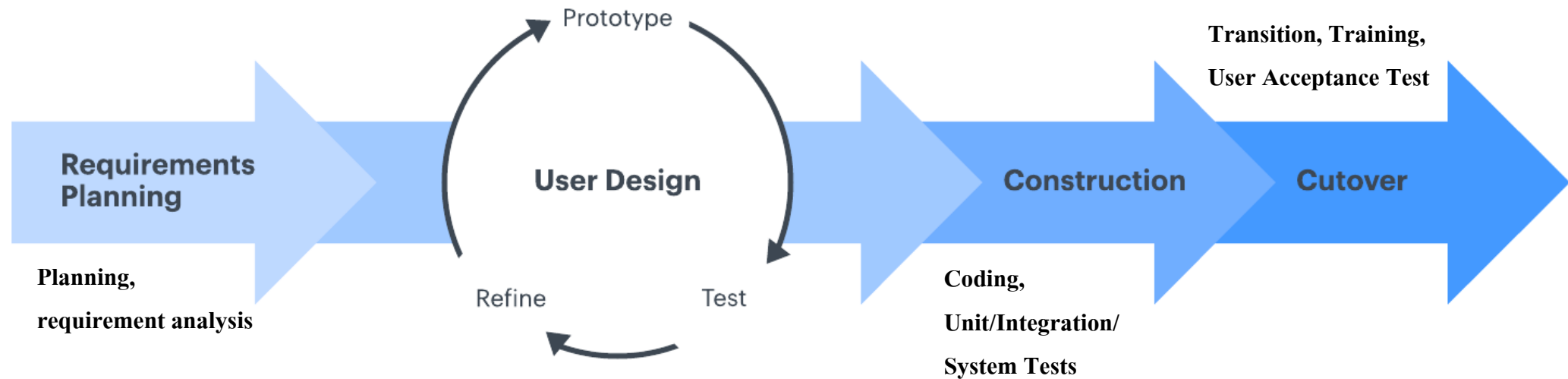
- Are we building the software right?
(เราได้สร้างซอฟต์แวร์อย่างถูกต้องหรือไม่?)
- เป็นการตรวจสอบว่าได้ทำถูกต้องตาม
ข้อกำหนด
- ทีมพัฒนาดำเนินการ

Validation

- Are we building the right software?
(เราได้สร้างซอฟต์แวร์ที่ถูกต้องหรือไม่)
- เป็นการพิสูจน์ว่าได้ทำในสิ่งที่ถูกต้องตาม
ความต้องการของลูกค้า
- ลูกค้าดำเนินการ

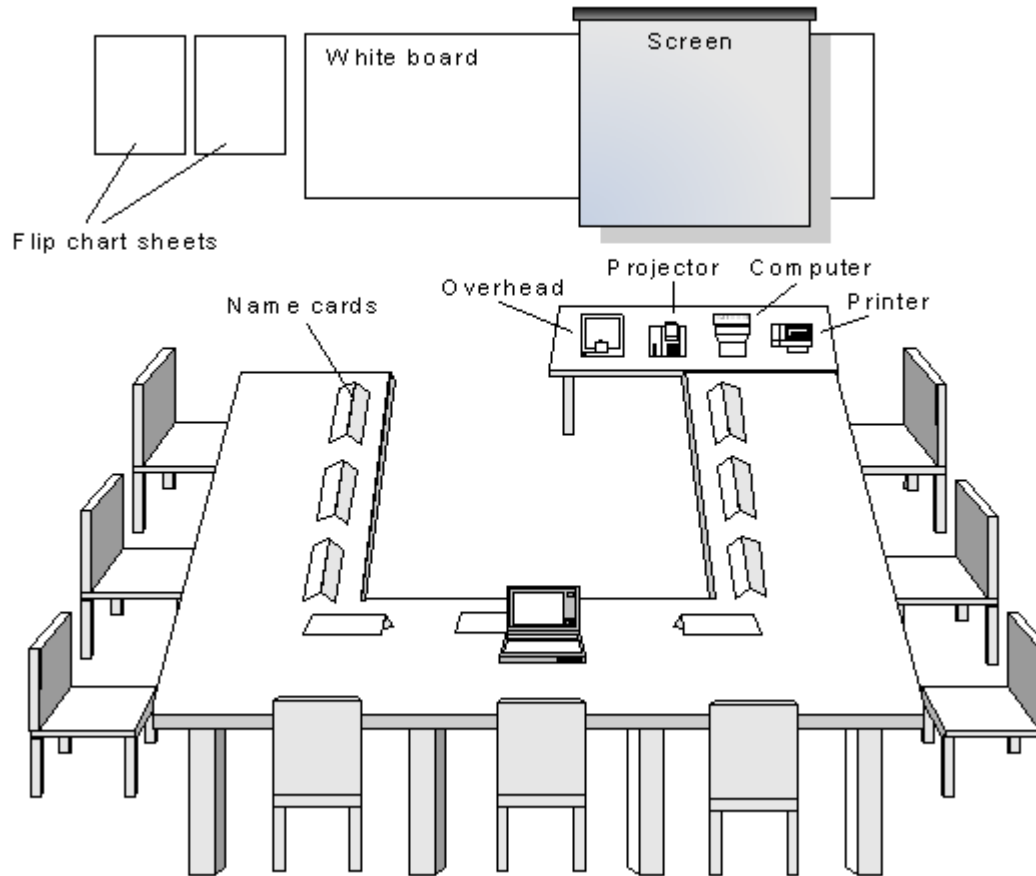
4. Rapid Application Development (RAD)

- ลดเวลาในการวางแผน เน้นการใช้ Prototype ในการสื่อสารกับผู้ใช้งาน ซึ่งเป็นการแก้ไขปัญหของ Waterfall ที่สามารถทำให้ผู้ใช้งานเห็นรูปร่างหน้าตาของซอฟต์แวร์ตั้งแต่ต้น
- ทีมพัฒนาและผู้ใช้งานร่วมกันพัฒนา Prototype มักใช้แนวทาง Joint Application Design (JAD) นั่นคือ ทีมพัฒนาและผู้ใช้งานทำงานร่วมกันในห้องปฏิบัติการ
- ผู้ใช้งานมีส่วนร่วมในระหว่างการ Coding โดยให้คำแนะนำจาก Screen หรือ Report ที่พัฒนาแล้ว



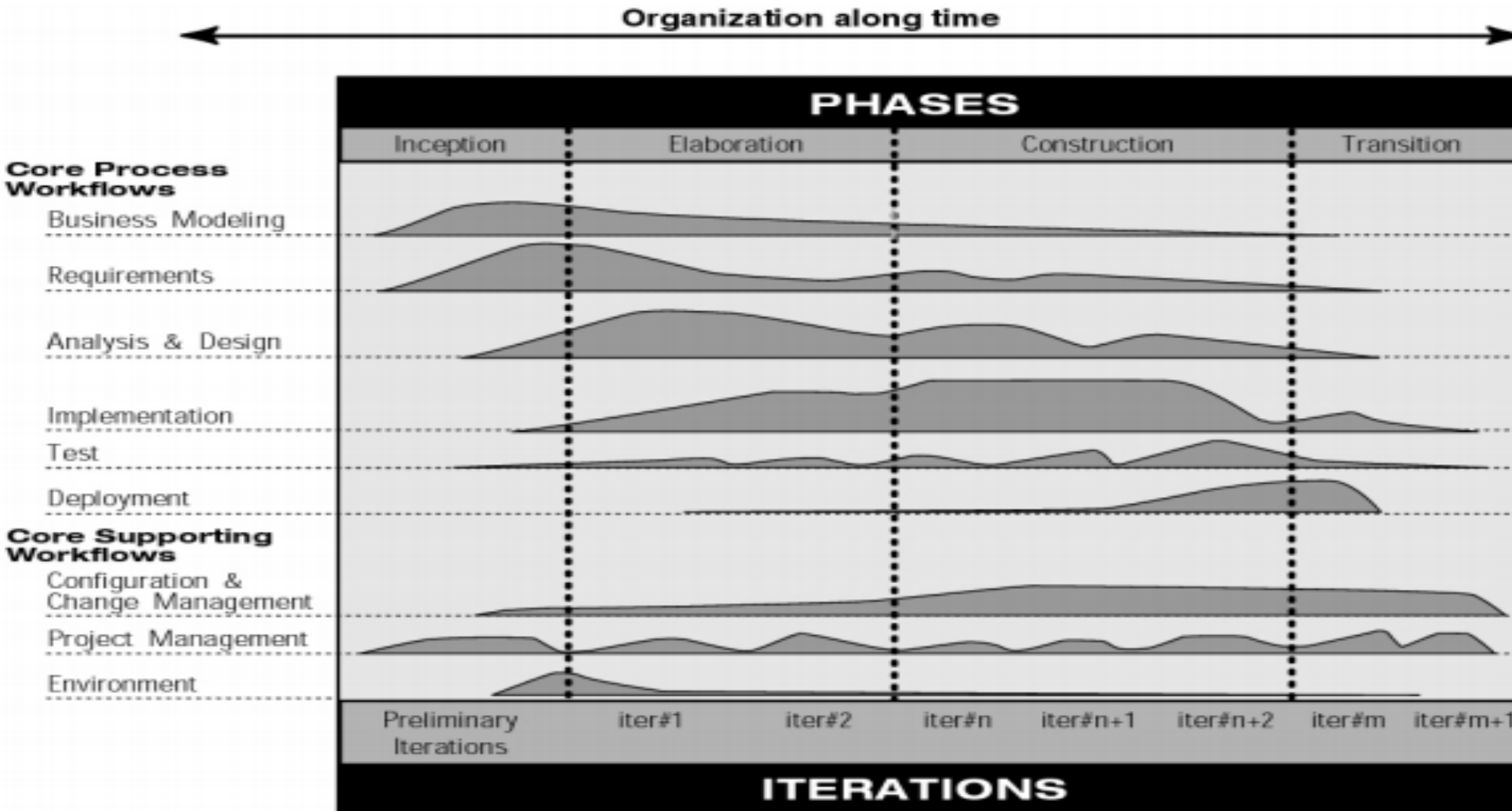
ภาพที่ 4-1 : RAD Model

ทีมพัฒนาและผู้ใช้งานทำงาน
ร่วมกันในห้องปฏิบัติการ
ด้วยความมุ่งมั่น



ภาพที่ 4-2 : ห้องปฏิบัติการสำหรับ Joint Application Design

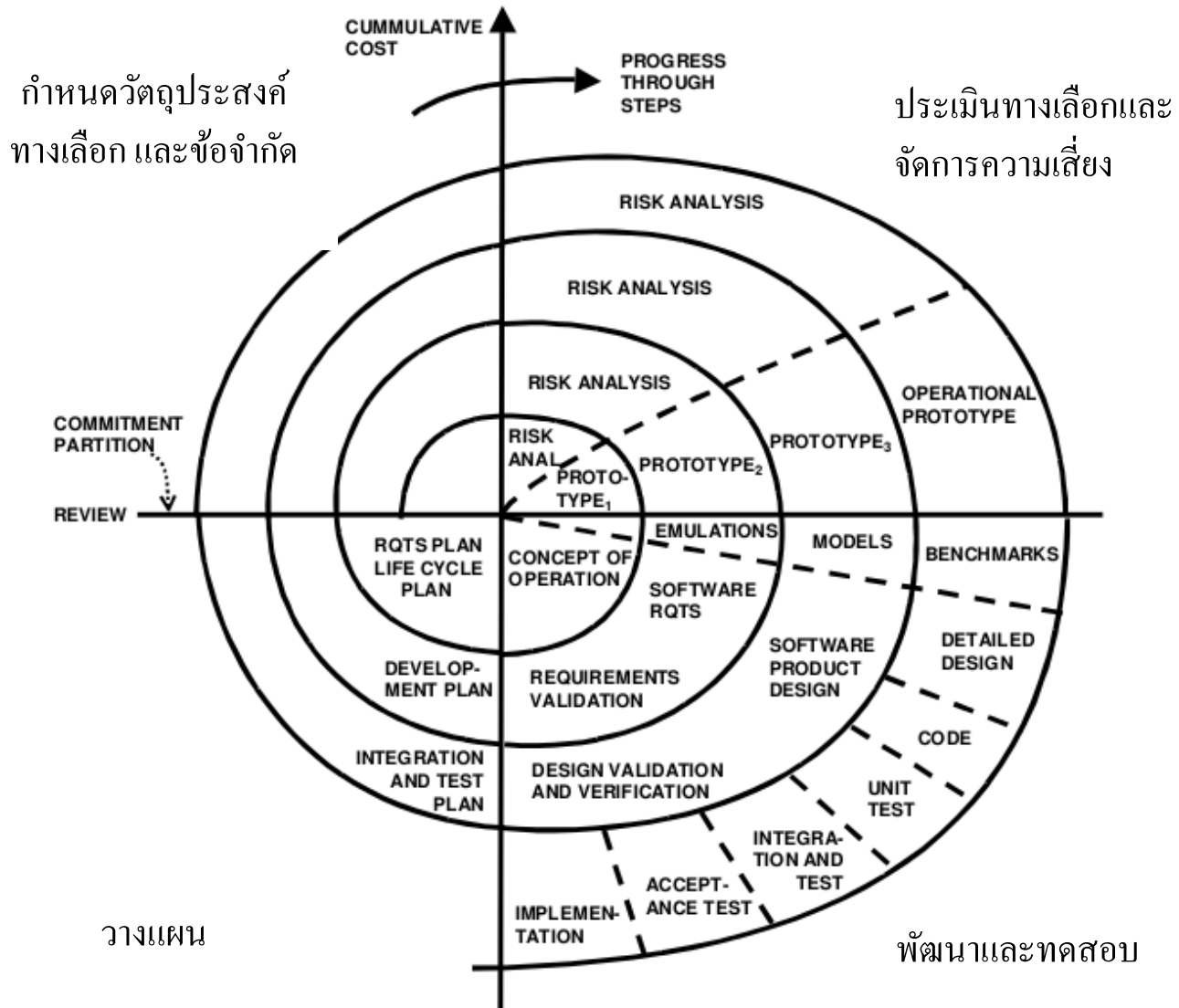
5. Rational Unified Process (RUP)



ภาพที่ 5-1 : RUP Model

- พัฒนาซอฟต์แวร์แบบวนรอบ (Iterative Development)
- ออกแบบไม่ละเอียดตั้งแต่แรก จากนั้นเพิ่มเติมความต้องการของผู้ใช้ภายหลัง

6. Spiral Model



ภาพที่ 6-1 : Spiral Model

- พัฒนาซอฟต์แวร์แบบวนรอบ
(Iterative Development)
- มีการจัดการความเสี่ยงทุกรอบ
- เหมาะกับการพัฒนาซอฟต์แวร์ที่มีความต้องการไม่ชัดเจน
- ใช้เวลานานและค่าใช้จ่ายสูงในการจัดการความเสี่ยง

การกำหนดวัตถุประสงค์ ทางเลือก และข้อจำกัด

- รวบรวมความต้องการ
- กำหนดวัตถุประสงค์ของความต้องการ
- กำหนดทางเลือกในการดำเนินการ
- ระบุข้อจำกัด

การประเมินทางเลือกและการจัดการความเสี่ยง

- ประเมินทางเลือก
- ระบุความเสี่ยง
- วิเคราะห์ความเสี่ยง
- แก้ไขความเสี่ยง

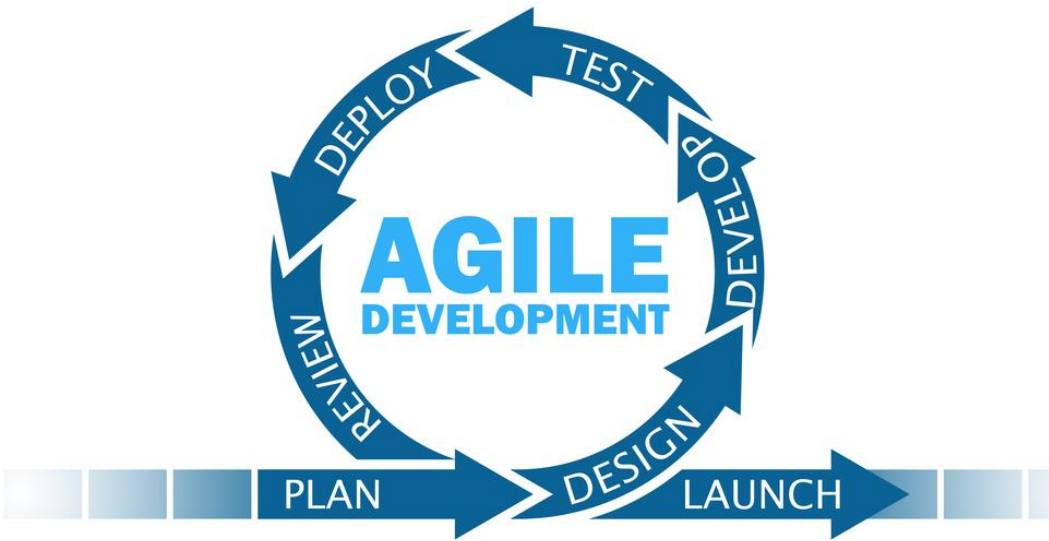
การวางแผน

- ตรวจสอบว่าได้ดำเนินงานตามแผนหรือไม่
- กำหนดแผนสำหรับรอบถัดไป

พัฒนาและทดสอบ

- พัฒนา
- ทดสอบ

7. Agile Model



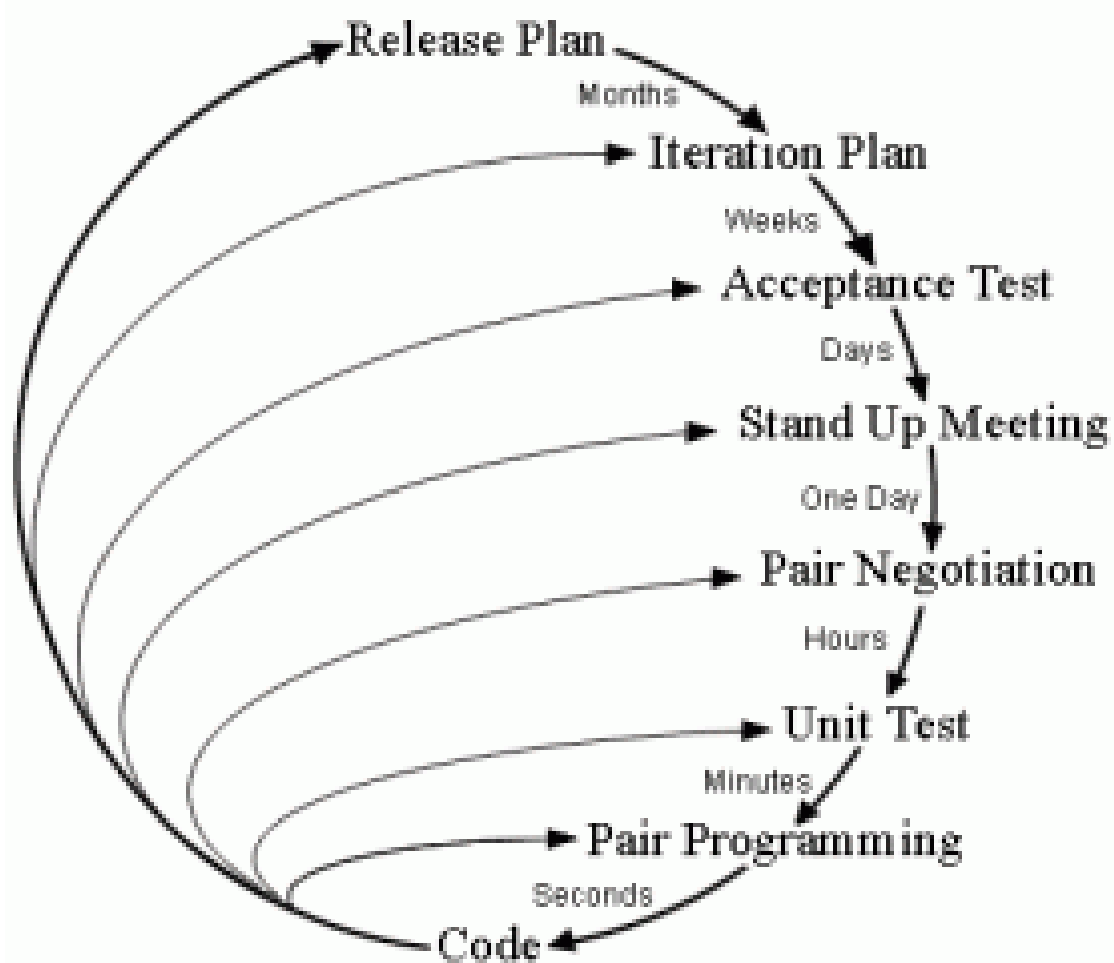
ภาพที่ 7-1 : Agile Model

- มีรูปแบบการพัฒนาแบบ **Iterative and Incremental Dev.** ซึ่งแต่ละรอบจะได้ส่วนของซอฟต์แวร์เพิ่มขึ้น
- ลดความเสียหายที่จะเกิดจากความล้มเหลวของโครงการ
- ลดความต้องการด้านเอกสาร
- ยอมรับในการเปลี่ยนแปลงความต้องการ
- ผู้ใช้งานถือเป็นส่วนหนึ่งของทีมพัฒนา ซึ่งต้องให้เวลาให้กับทีมมากพอ

Agile มีบทบาทบัญญัติ 4 ประการที่ต้องคำนึงเมื่อต้องพัฒนาซอฟต์แวร์ ดังนี้ [Agile Alliance 2001]

- ทีมงานทุกคนมีคุณค่าในตัวเองมากพอที่จะทำงานอย่างอิสระได้ นั่นคือทีมงานใช้ทักษะที่ตัวเองมีในการพัฒนาซอฟต์แวร์เพื่อให้ตรงตามความต้องการของลูกค้า มากกว่าจะถูกจำกัดวิธีการพัฒนาด้วยกระบวนการหรือเครื่องมือ (Result-Oriented)
- ทีมงานพอใจที่จะใช้เวลาส่วนใหญ่ในการเขียนโปรแกรมเพื่อสร้างซอฟต์แวร์มากกว่าการใช้เวลาเพื่อจัดทำเอกสาร (Less Documents)
- ทีมงานมุ่งเน้นการทำงานร่วมกับลูกค้าโดยตรง แทนการเจรจาอย่างเป็นทางการตามสัญญาว่าจ้าง (Informal Communication)
- ทีมงานให้ความสำคัญกับการแก้ไขงานทันทีที่มีการเปลี่ยนแปลง (Rapid Change)

Extreme Programming



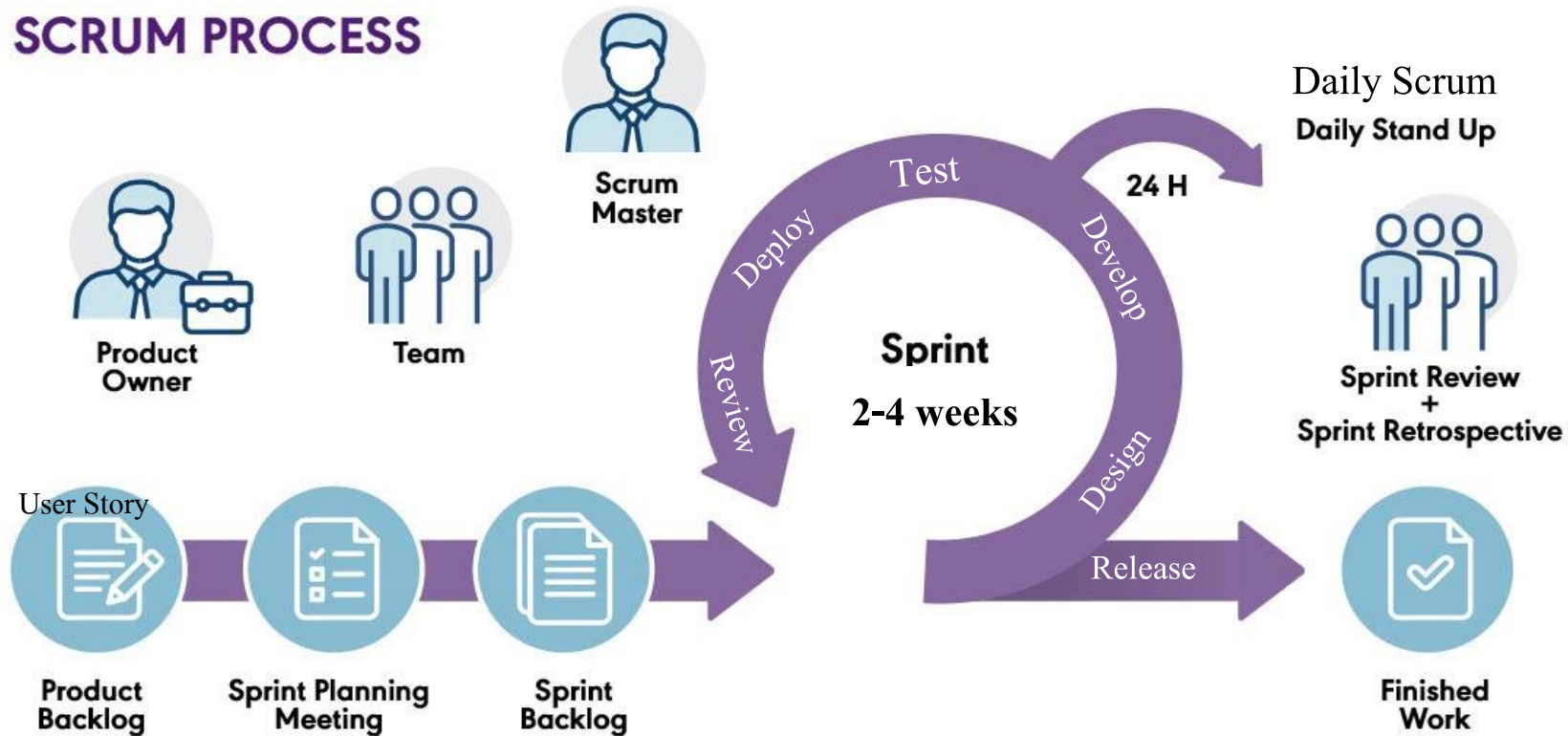
ภาพที่ 7-2 : Extreme Programming Model

- เป็นรูปแบบหนึ่งของ Agile Development
- เน้นที่ขั้นตอน Coding (มี Pair Programming)
- สร้าง Test Case ก่อนการ Coding (Test-Case First)
- มีการทำ Code Refactoring เพื่อให้ง่ายต่อการเปลี่ยนแปลงความต้องการ

Scrum Development

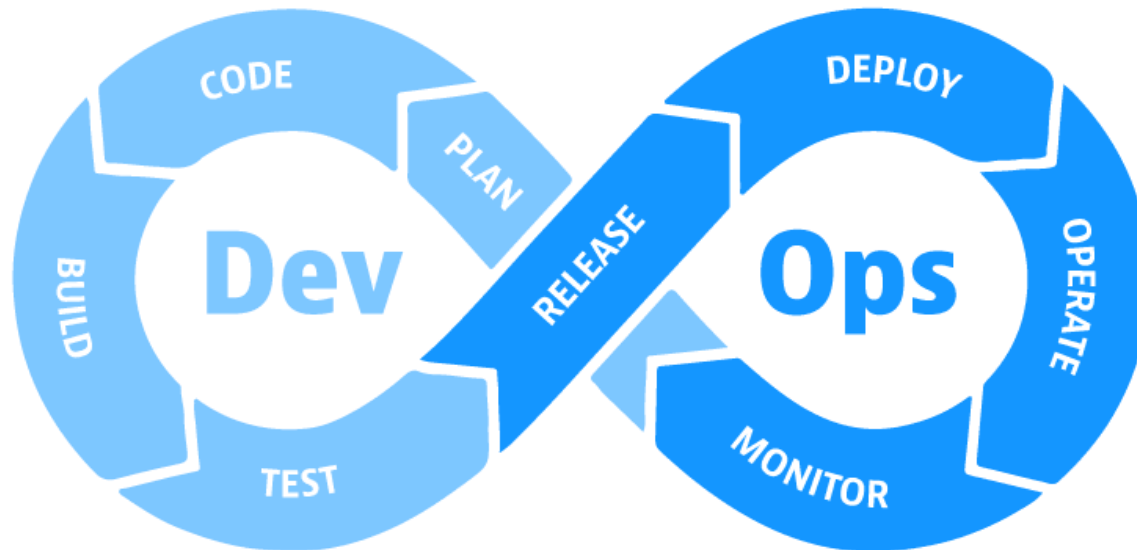
- เป็นรูปแบบหนึ่งของ Agile Development
- มีการกำหนดรอบของการส่งมอบงานในช่วงเวลาสั้น ๆ เรียกว่า Sprint ที่ 1 Sprint ใช้เวลา 2-4 สัปดาห์
- มีการติดตามงานและปรับแผนทุกวันเรียกว่า Daily Scrum หรือ Stand-up Meeting
- มีทีมพัฒนาขนาดเล็กประมาณ 3-9 คน

SCRUM PROCESS



8. DevOps

- เป็นการรวมกันของ Software Development และ Operation
- เป็นส่วนเสริมของ Agile Development
- Continuous Integration/Continuous Delivery (CI/CD)



Do you have any questions?

